

Lab - 3:

Experiment No: 3

Dated:

AIM: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Lab Objectives: To implement public and private Blockchain.

Lab Outcomes (LO): Demonstrate the concept of Blockchain in real-world Applications (LO4)

Task to be performed :

1. Download the code from folder, [Lab_3](#)
2. Install requests in the virtual environment created in the Lab 2. ([Follow the instructions](#))
3. Run the files - **hadcoin_node_5001.py**, **hadcoin_node_5002.py**, **hadcoin_node_5003.py** in 3 different terminals.
4. Open Postman, from each node - invoke **connect_node()** and pass the peers as POST requests.
5. Perform the following functions
 - Add Transactions - invoke **add_transactions()** as a POST request.
 - mining - **mine_block()**,
 - fetch the chain - **get_chain()**,
 - replace the longest chain - **replace_chain()**
6. Modify the code such that transactions are removed after they are added to the block.

Tools & Libraries used :

- Install [Flask](#) : pip install Flask
- Download **Postman** from <https://www.postman.com/>
- Python Libraries : **datetime, jsonify, hashlib, uuid4, urlparse, request**
- **Install requests** : pip install requests==2.18.4

Instructions : (Prepare for viva for the following topics)

1. Challenges in P2P networks
2. How transactions are performed on the network?
3. Explain the role of mempools
4. Write briefly about the libraries and the tools used during implementation.

Outcome :

1. Understood the challenges in P2P networks, how transactions are performed and how a miner mines a block to be added in a blockchain.
2. Implemented a Cryptocurrency in Python using Flask, Postman and Python libraries such as datetime, jsonify, hashlib, uuid4, urlparse, request.
3. Successfully mined the blocks among a P2P network with 3 nodes.
4. Performed transactions via the network.
5. Successfully updated the block across the network
6. Prepare a document with Aim, Tasks performed, Program, Output and Conclusion.

7. Submit the hardcopy **by Sem** (As per the instructions, submit a hard copy of the same).

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Guidelines

Lab Objectives: To implement public and private Blockchain.

Lab Outcomes (LO): Demonstrate the concept of Blockchain in real-world Applications (LO4)

Task to be performed :

1. Download the code from folder, Lab_3
2. Install requests in the virtual environment created in the Lab 2. (Follow the instructions)
3. Run the files - hadcoin_node_5001.py, hadcoin_node_5002.py, hadcoin_node_5003.py in 3 different terminals.
4. Open Postman, from each node - invoke connect_node() and pass the peers as POST requests.
5. Perform the following functions
 - Add Transactions - invoke add_transactions() as a POST request.
 - mining - mine_block(),
 - fetch the chain - get_chain(),
 - replace the longest chain - replace_chain()
6. Modify the code such that transactions are removed after they are added to the block.

Tools & Libraries used :

- Install Flask : pip install Flask
- Download Postman from <https://www.postman.com/>
- Python Libraries : datetime, jsonify, hashlib, uuid4, urlparse, request
- Install requests : pip install requests==2.18.4

Instructions : (Prepare for viva for the following topics)

1. Challenges in P2P networks
2. How transactions are performed on the network?
3. Explain the role of mempools
4. Write briefly about the libraries and the tools used during implementation.

Outcome :

1. Understood the challenges in P2P networks, how transactions are performed and how a

miner mines a block to be added in a blockchain.

2. Implemented a Cryptocurrency in Python using Flask, Postman and Python libraries such as datetime, jsonify, hashlib, uuid4, urlparse, request.
3. Successfully mined the blocks among a P2P network with 3 nodes.
4. Performed transactions via the network.
5. Successfully updated the block across the network
6. Prepare a document with Aim, Tasks performed, Program, Output and Conclusion.
7. Submit the hardcopy by the 2nd week of August 2023
(As per the instructions, submit a hard copy of the same).

Theory:

1. Blockchain Overview

Blockchain is a **distributed and decentralized ledger** that stores information in a series of linked blocks.

Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain, it becomes **immutable** because altering one block would require recalculating all subsequent blocks.

2. Mining

Mining is the process of:

1. Collecting pending transactions into a block.
2. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
3. Adding the new block to the blockchain.
Broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this lab, we simulate **three independent blockchain nodes** (5001, 5002, 5003).

Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Can connect with peers to share and validate blocks.

4. Consensus Mechanism

We use the **Longest Chain Rule**:

- If multiple versions of the chain exist, the **longest valid chain** is chosen.
- This ensures all nodes agree on a single transaction history.

5. Transactions & Mining Reward

Each transaction has:

- Sender
- Receiver
- Amount

When mining a block:

- Pending transactions are added to the block.
- A **reward transaction** is added automatically to pay the miner.

6. Chain Replacement

When `/replace_chain` is called:

1. Node requests chains from peers.
2. If it finds a longer and valid chain, it replaces its own.
3. This keeps the blockchain consistent across all nodes.

Tools & Libraries Used

- **Python 3.x**
- **Flask** – Web framework for API endpoints
`pip install Flask`
- **Requests** – For HTTP communication between nodes
`pip install requests==2.18.4`
- **Postman** – For testing API requests
- Python Standard Libraries:
 - `datetime`
 - `jsonify`
 - `hashlib`
 - `uuid4`
 - `urlparse`
 - `request`

Procedure and Output:

1. Download `hadcoin_node_5001.py`, `hadcoin_node_5002.py`, `hadcoin_node_5003.py`.
2. Install required packages.
3. Run each node in separate terminals.
4. Connect nodes using Postman – POST `/connect_node`.

5.

6. Add transactions – POST
7. Add 3 transactions in 5001
8. Condition 2 — Each node must have **different chain lengths**

Example:

If you mined 1 block on 5001, but did not mine on 5002 or 5003 →

- 5001 chain length = 2
- 5002 chain length = 1
- 5003 chain length = 1

STEP 2 — Perform all transactions ONLY from one node (e.g., 5001)

POST → http://127.0.0.1:5001/add_transaction

```
{  
  
  "sender": "a",  
  
  "receiver": "b",  
  
  "amount": 5
```

```
}
```

Then 5001 → GET `/mine_block`

STEP 3 — Now go to 5002 and run:

GET → `/replace_chain`

STEP 4 — Repeat on 5003

GET → `/replace_chain`

Same result.

```
{  
  
  "sender": "a",  
  
  "receiver": "b",  
  
  "amount": 53  
}
```

Output

```
{  
  
  "index": 3,  
  
  "message": "Congratulations, you just mined a block!",  
}
```

```
"previous_hash":
"d501f26a58a2bfccd173a1da675769cae717df3b4536e0f2698fe8d27a45436b",

"proof": 45293,

"timestamp": "2026-01-28 15:34:33.283321",

"transactions": [

  {

    "amount": 51,

    "receiver": "b",

    "sender": "a"

  },

  {

    "amount": 52,

    "receiver": "b",

    "sender": "a"

  },

  {

    "amount": 53,

    "receiver": "b",

    "sender": "a"

  },

  {

    "amount": 1,

    "receiver": "Richard",

    "sender": "609febafd5104eafa33aech4de9ea7b6"
```

```
}  
  
1  
  
}
```

9. Mine blocks – GET `/mine_block`.

The screenshot shows a REST client interface with the following details:

- URL:** `http://127.0.0.1:5001/mine_block`
- Method:** `GET`
- Query Params:** A table with 2 columns: Key and Value.
- Status:** `200 OK`, Time: `21 ms`, Size: `465 B`
- Body:** A JSON object representing a mined block.

Key	Value
Key	Value

```
{  
  "index": 10,  
  "message": "Congratulations, you just mined a block!",  
  "previous_hash": "31dfc2d4b483b532a49f5920f9c4cf874124c8ce1cd40d07f0872e67da2bda0e",  
  "proof": 15479,  
  "timestamp": "2026-08-31 12:02:05.254384",  
  "transactions": [{"amount": 1, "receiver": "Richard", "sender": "4151c25e2d1c403cba14d56701565993"}]  
}
```

10. Fetch blockchain – GET `/get_chain`.

HTTP client interface showing a GET request to `http://127.0.0.1:5003/get_chain`. The response is a JSON array of 6 blocks in a blockchain chain.

```
1 [{"chain": [{"index": 1, "previous_hash": "0", "proof": 1, "timestamp": "2026-08-31 11:39:31.601975", "transactions": []}, {"index": 2, "previous_hash": "078968aca2a3492cc6b1cf8f6eff4b0c5a674c18951c2308acf6873f4405c5f0", "proof": 533, "timestamp": "2026-08-31 11:45:18.821739", "transactions": [{"amount": 51, "receiver": "b", "sender": "a"}, {"amount": 52, "receiver": "b", "sender": "a"}, {"amount": 53, "receiver": "b", "sender": "a"}, {"amount": 1, "receiver": "Richard", "sender": "4151c25e2d1c403cba14d56701565993"}]}, {"index": 3, "previous_hash": "09a4126173ff5cc5d08822d2810434b4906f68552ace36ab443279e171016fcc", "proof": 45293, "timestamp": "2026-08-31 11:47:33.035706", "transactions": [{"amount": 53, "receiver": "Shourya", "sender": "Chinmay"}, {"amount": 52, "receiver": "Shourya", "sender": "Chinmay"}, {"amount": 51, "receiver": "Shourya", "sender": "Chinmay"}, {"amount": 1, "receiver": "Richard", "sender": "4151c25e2d1c403cba14d56701565993"}]}, {"index": 4, "previous_hash": "4037f0587435bdd9112b399dcf878c451c1adeba8e70b3e3f3113df48b01cef3", "proof": 21391, "timestamp": "2026-08-31 11:47:42.305346", "transactions": [{"amount": 1, "receiver": "Richard", "sender": "4151c25e2d1c403cba14d56701565993"}]}, {"index": 5, "previous_hash": "f137cec0506a0b62827831c2c17e0f05a1ed0177418708f48ead90552658d681", "proof": 8018, "timestamp": "2026-08-31 11:48:16.834086", "transactions": [{"amount": 1, "receiver": "Richard", "sender": "4151c25e2d1c403cba14d56701565993"}]}, {"index": 6, "previous_hash": "c493a8b536bc2b7b555e988f3288a8b1a8999248873a9d8936d8f32f27873718", "proof": 48191, "timestamp": "2026-08-31 11:50:23.618997", "transactions": [{"amount": 51, "receiver": "Shourya", "sender": "Chinmay"}, {"amount": 52, "receiver": "Shourya", "sender": "Chinmay"}, {"amount": 1, "receiver": "Richard", "sender": "4151c25e2d1c403cba14d56701565993"}]}], "length": 6}]
```

11. Replace chain – GET `/replace_chain`.

HTTP client interface showing a GET request to `http://127.0.0.1:5002/replace_chain`. The response is a JSON object with a message and a new chain.

```
1 [{"message": "The nodes had different chains so the chain was replaced by the longest one.", "new_chain": [{"index": 1, "previous_hash": "0", "proof": 1, "timestamp": "2026-08-31 11:39:31.601975", "transactions": []}, {"index": 2, "previous_hash": "078968aca2a3492cc6b1cf8f6eff4b0c5a674c18951c2308acf6873f4405c5f0", "proof": 533, "timestamp": "2026-08-31 11:45:18.821739", "transactions": [{"amount": 51, "receiver": "b", "sender": "a"}, {"amount": 52, "receiver": "b", "sender": "a"}, {"amount": 53, "receiver": "b", "sender": "a"}, {"amount": 1, "receiver": "Richard", "sender": "4151c25e2d1c403cba14d56701565993"}]}, {"index": 3, "previous_hash": "09a4126173ff5cc5d08822d2810434b4906f68552ace36ab443279e171016fcc", "proof": 45293, "timestamp": "2026-08-31 11:47:33.035706", "transactions": [{"amount": 53, "receiver": "Shourya", "sender": "Chinmay"}, {"amount": 52, "receiver": "Shourya", "sender": "Chinmay"}, {"amount": 51, "receiver": "Shourya", "sender": "Chinmay"}, {"amount": 1, "receiver": "Richard", "sender": "4151c25e2d1c403cba14d56701565993"}]}, {"index": 4, "previous_hash": "4037f0587435bdd9112b399dcf878c451c1adeba8e70b3e3f3113df48b01cef3", "proof": 21391, "timestamp": "2026-08-31 11:47:42.305346", "transactions": [{"amount": 1, "receiver": "Richard", "sender": "4151c25e2d1c403cba14d56701565993"}]}, {"index": 5, "previous_hash": "f137cec0506a0b62827831c2c17e0f05a1ed0177418708f48ead90552658d681", "proof": 8018, "timestamp": "2026-08-31 11:48:16.834086", "transactions": [{"amount": 1, "receiver": "Richard", "sender": "4151c25e2d1c403cba14d56701565993"}]}, {"index": 6, "previous_hash": "c493a8b536bc2b7b555e988f3288a8b1a8999248873a9d8936d8f32f27873718", "proof": 48191, "timestamp": "2026-08-31 11:50:23.618997", "transactions": [{"amount": 51, "receiver": "Shourya", "sender": "Chinmay"}, {"amount": 52, "receiver": "Shourya", "sender": "Chinmay"}, {"amount": 1, "receiver": "Richard", "sender": "4151c25e2d1c403cba14d56701565993"}]}]}
```

12.

HTTP

http://127.0.0.1:5001/mine_block

Save

No Environment

GET

http://127.0.0.1:5001/mine_block

Send

Params

Authorization

Headers (7)

Body

Scripts

Settings

Code

Cookies

Query Params

	Key	Value
	Key	Value

Body

Cookies

Headers (5)

Test Results

Status: 200 OK

Time: 53 ms

Size: 572 B

...

+

-

Pretty

Raw

Preview

JSON

↻

📄

🔍

1

2

```
[{"index":6,"message":"Congratulations, you just mined a block!",
  "previous_hash":"c493a8b536bc2b7b555e988f3288a8b1a8999248873a9d8936d8f32f27873718","proof":48191,
  "timestamp":"2026-08-31 11:50:23.618997","transactions":[{"amount":51,"receiver":"Shourya",
    "sender":"Chinmay"},{"amount":52,"receiver":"Shourya","sender":"Chinmay"},{"amount":1,
    "receiver":"Richard","sender":"4151c25e2d1c403cba14d56701565993"}]}]
```

127.0.0.1:5001/get_chain

📄

🌟

🔍

👤

...

Chat

Pretty-print

🔍

```
{
  "index": 3,
  "previous_hash": "09a4126173ff5cc5d08822d2810434b4906f68552ace36ab443279e171016fcc",
  "proof": 45293,
  "timestamp": "2026-08-31 11:47:33.035706",
  "transactions": [
    {
      "amount": 53,
      "receiver": "Shourya",
      "sender": "Chinmay"
    },
    {
      "amount": 52,
      "receiver": "Shourya",
      "sender": "Chinmay"
    },
    {
      "amount": 51,
      "receiver": "Shourya",
      "sender": "Chinmay"
    },
    {
      "amount": 1,
      "receiver": "Richard",
      "sender": "4151c25e2d1c403cba14d56701565993"
    }
  ]
},
{
  "index": 4,
  "previous_hash": "4037f0587435bbd9112b399dcf878c451c1adeba8e70b3e3f3113df48b01cef3",
  "proof": 21391,
  "timestamp": "2026-08-31 11:47:42.305346",
  "transactions": [
    {
      "amount": 1,
      "receiver": "Richard",
      "sender": "4151c25e2d1c403cba14d56701565993"
    }
  ]
}
]
```

```
← 127.0.0.1:5002/replace_chain
Pretty-print ☒
},
{
  "index": 5,
  "previous_hash": "f137cec0506a0b62827831c2c17e0f05a1ed0177418708f48ead90552658d681",
  "proof": 8018,
  "timestamp": "2026-08-31 11:48:16.834086",
  "transactions": [
    {
      "amount": 1,
      "receiver": "Richard",
      "sender": "4151c25e2d1c403cba14d56701565993"
    }
  ]
},
{
  "index": 6,
  "previous_hash": "c493a8b536bc2b7b555e988f3288a8b1a8999248873a9d8936d8f32f27873718",
  "proof": 48191,
  "timestamp": "2026-08-31 11:50:23.618997",
  "transactions": [
    {
      "amount": 51,
      "receiver": "Shourya",
      "sender": "Chinmay"
    },
    {
      "amount": 52,
      "receiver": "Shourya",
      "sender": "Chinmay"
    },
    {
      "amount": 1,
      "receiver": "Richard",
      "sender": "4151c25e2d1c403cba14d56701565993"
    }
  ]
},
{
  "message": "All good. The chain is the largest one."
}
```

Conclusion:

We developed a cryptocurrency using Python and implemented mining in a simulated three-node blockchain network. Each node maintained its own ledger and synchronized with peers using the Longest Chain Rule to ensure consistency. Mining was performed through Proof-of-Work, securely adding transactions and rewarding miners. This demonstrated key blockchain concepts, including decentralized consensus, transaction validation, and network synchronization.

